

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

Assessment Tool Weightage: 40%

QUESTIONS:5

Duration :60 Mins
On Class
Total Marks:50

Annexure A

EXPERIMENTS

Code of Conduct:

*I Arunprasath(192524077) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Arunprasath.R

Annexure A

EXPERIMENTS

Q1. Write a C program to implement a syntax-directed definition for evaluating arithmetic expressions using the following grammar:

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow \text{num}$$

Input:

3 + 5 * 2

The program should display the attribute values and the final value of the expression.
(10 Marks)

ANS:

Aim

To write a C program to implement a syntax-directed definition for evaluating arithmetic expressions using the grammar:

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow \text{num}$$

Input

3 + 5 * 2

Theory

For the given grammar:

$$F \rightarrow \text{num}$$

the value of the number is assigned to F.val.

For multiplication:

$$T \rightarrow T * F$$

the value is calculated as:

$$T.val = T1.val * F.val$$

For addition:

$$E \rightarrow E + T$$

the value is calculated as:

$$E.val = E1.val + T.val$$

Since multiplication has higher precedence than addition:

$$3 + 5 * 2$$

$$= 3 + (5 * 2)$$

$$= 3 + 10$$

$$= 13$$

Algorithm

1. Start the program.
2. Assign the input numbers to the leaf attributes.
3. Calculate the value of $5 * 2$.
4. Assign the result to $T.val$.
5. Calculate $3 + T.val$.
6. Assign the result to $E.val$.
7. Display all attribute values.
8. Display the final expression value.
9. Stop.

Result

Thus, the syntax-directed definition was successfully implemented in C, and the expression $3 + 5 * 2$ was evaluated correctly as **13**.

```

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>dir exp_q1.c
Volume in drive C is OS
Volume Serial Number is BC6A-BF46

Directory of C:\Users\arunp\OneDrive\Documents\Compiler - LAB

13-08-2026  11:20                964 exp_q1.c
             1 File(s)              964 bytes
             0 Dir(s)  325,729,333,248 bytes free

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>set PATH=%PATH%;C:\MinGW\bin

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>gcc exp_q1.c -o exp_q1.exe

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>exp_q1.exe
SYNTAX-DIRECTED DEFINITION
=====
Input: 3 + 5 * 2

Attribute Values:
F1.val = 3
F2.val = 5
F3.val = 2

T.val = F2.val * F3.val
T.val = 5 * 2
T.val = 10

E.val = E1.val + T.val
E.val = 3 + 10
E.val = 13

Final Value of Expression = 13

```

Q2. Write a C program to implement a simple type checker for arithmetic expressions using the following grammar:

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow \text{id}$$

Consider the following symbol table:

a : int

b : int

c : float

Input:

a + b * c

Determine the type of each subexpression and the final expression. (10 Marks)

ANS:

Type analysis

b : int

c : float

b * c : float

Since int and float are combined, the integer operand is promoted to float.

Then:

a : int

b * c : float

a + (b * c) : float

Therefore:

Final expression type = float

Result

The simple type checker was successfully implemented for the given grammar. For the expression $a + b * c$, the subexpression $b * c$ has type float, and the complete expression $a + (b * c)$ also has type float. Hence, the final type of the expression is **float**.

```
C:\Users\arunp\OneDrive\Documents\Compiler - LAB>dir exp_q2.c
Volume in drive C is OS
Volume Serial Number is BC6A-BF46

Directory of C:\Users\arunp\OneDrive\Documents\Compiler - LAB

13-08-2026  11:22                1,347 exp_q2.c
             1 File(s)              1,347 bytes
             0 Dir(s)  325,734,744,064 bytes free

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>gcc exp_q2.c -o exp_q2.exe

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>exp_q2.exe
SIMPLE TYPE CHECKER
=====
Symbol Table:
a : int
b : int
c : float

Input Expression: a + b * c

Step 1: Type checking b * c
b : int
c : float
b * c : float

Step 2: Type checking a + (b * c)
a : int
b * c : float
a + (b * c) : float

Final Expression Type: float
```

Q3. Write a C program to determine whether two type expressions are equivalent.

Consider the following type expressions:

T1 = array(10, int)

T2 = array(10, int)

The program should display:

Equivalent or Not Equivalent . (10 Marks)

ANS:

Aim

To write a C program to determine whether two type expressions are equivalent.

Given:

T1 = array(10, int)

T2 = array(10, int)

Theory

Type equivalence determines whether two type expressions represent the same type structure.

For structural equivalence, the corresponding components of the type expressions are compared.

Given:

T1 = array(10, int)

T2 = array(10, int)

The following are compared:

- Type constructor → array
- Array size → 10
- Element type → int

Since all corresponding components are identical, the two type expressions are equivalent.

Algorithm

1. Start the program.
2. Create structures to represent the two type expressions.
3. Store the constructor, size and element type.
4. Compare the constructors.
5. Compare the array sizes.
6. Compare the element types.
7. If all are equal, display Equivalent.
8. Otherwise, display Not Equivalent.
9. Stop.

Result

Thus, the two type expressions `array(10, int)` and `array(10, int)` were successfully compared and found to be **Equivalent**.

```
C:\Users\arunp\OneDrive\Documents\Compiler - LAB>dir exp_q3.c
Volume in drive C is OS
Volume Serial Number is BC6A-BF46

Directory of C:\Users\arunp\OneDrive\Documents\Compiler - LAB

13-08-2026  11:25                981 exp_q3.c
             1 File(s)              981 bytes
             0 Dir(s) 325,734,293,504 bytes free

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>gcc exp_q3.c -o exp_q3.exe

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>exp_q3.exe
TYPE EXPRESSION EQUIVALENCE
=====
T1 = array(10, int)
T2 = array(10, int)

Checking type expressions...

Result: Equivalent
```

Q4. Write a C program to construct and display the syntax tree for the following expression using the given grammar:

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow \text{id}$$

Input:

`a + b * c`

Display the constructed syntax tree. (10 Marks)

ANS:

Aim

To write a C program to construct and display the syntax tree for the expression:

$a + b * c$

using the grammar:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow \text{id}$

Theory

A syntax tree represents the hierarchical structure of an expression.

The grammar gives multiplication higher precedence than addition. Therefore:

$a + b * c$

is interpreted as:

$a + (b * c)$

The corresponding syntax tree is:

```

      +
     /\
    a  *
     /\
    b  c
```

The + operator becomes the root, while * becomes the right child of +.

Algorithm

1. Start the program.
2. Create a node for +.
3. Create a node for a.
4. Create a node for *.
5. Create nodes for b and c.
6. Make a the left child of +.

7. Make * the right child of +.
8. Make b and c the children of *.
9. Traverse and display the syntax tree.
10. Stop.

Result

Thus, the syntax tree for $a + b * c$ was successfully constructed and displayed. The tree correctly represents $a + (b * c)$, following the precedence of multiplication over addition.

```
C:\Users\arunp\OneDrive\Documents\Compiler - LAB>dir exp_q4.c
Volume in drive C is OS
Volume Serial Number is BC6A-BF46

Directory of C:\Users\arunp\OneDrive\Documents\Compiler - LAB

13-08-2026  11:26                1,541 exp_q4.c
             1 File(s)              1,541 bytes
             0 Dir(s) 325,716,496,384 bytes free

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>gcc exp_q4.c -o exp_q4.exe

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>exp_q4.exe
SYNTAX TREE CONSTRUCTION
=====

Input Expression: a + b * c

Constructed Syntax Tree:

      c
     *
    b
   +
  a

Expression represented by tree:
a + (b * c)
```

Q5. Write a C program to demonstrate the propagation of inherited and synthesized attributes using the following grammar:

$D \rightarrow T L$

$T \rightarrow \text{int}$

$L \rightarrow \text{id } L'$

$L' \rightarrow , \text{id } L' \mid \epsilon$

Input:

int a,b,c

Display the type assigned to each identifier. (10 Marks)

ANS:

Aim

To write a C program to demonstrate the propagation of inherited and synthesized attributes using the grammar:

$$D \rightarrow T L$$
$$T \rightarrow \text{int}$$
$$L \rightarrow \text{id } L'$$
$$L' \rightarrow , \text{id } L' \mid \epsilon$$

Input:

int a,b,c

Theory

Attributes are associated with grammar symbols to carry semantic information during parsing.

In this declaration:

int a,b,c

the type int is obtained from:

$$T \rightarrow \text{int}$$

The type is then propagated to the identifier list.

Conceptually:

$$T.\text{type} = \text{int}$$

↓

$$L.\text{in} = \text{int}$$

↓

a, b, c

Therefore:

a : int

b : int

c : int

The type information passed from the parent/context to the identifier list is an **inherited attribute**.

Algorithm

1. Start the program.
2. Set the type from $T \rightarrow \text{int}$.
3. Pass the type as an inherited attribute to L.
4. Assign the inherited type to the first identifier.
5. Propagate the same type through L'.
6. Assign the type to the remaining identifiers.
7. Display the identifier-type table.
8. Stop.

Result

Thus, the propagation of attributes was successfully demonstrated. The type `int` is propagated from the type declaration to all identifiers in the declaration list. Hence:

```
a : int
b : int
c : int
```

were successfully obtained.

```
C:\Users\arunp\OneDrive\Documents\Compiler - LAB>dir exp_q5.c
Volume in drive C is OS
Volume Serial Number is BC6A-BF46

Directory of C:\Users\arunp\OneDrive\Documents\Compiler - LAB

13-08-2026  11:27                1,167 exp_q5.c
             1 File(s)              1,167 bytes
             0 Dir(s)  325,719,744,512 bytes free

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>gcc exp_q5.c -o exp_q5.exe

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>exp_q5.exe
INHERITED AND SYNTHESIZED ATTRIBUTES
=====
Input: int a,b,c

Step 1: Type Declaration
T.type = int

Step 2: Propagating inherited attribute
L.in = T.type
L.in = int

Step 3: Assigning type to identifiers
a.type = L.in = int
b.type = L.in = int
c.type = L.in = int

Identifier Type Table:
-----
a : int
b : int
c : int

Final Result:
All identifiers are assigned type int.
```

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation